

Monte Carlo Methods for Value-at-Risk and Expected Shortfall: Comparing Traditional and Deep Generative Scenario Simulations

Aneesh Chatterjee, Amily Li

May 16, 2025

This *Colab notebook* contains the implementation codes.

1 Introduction

Value at Risk (VaR) and Expected Shortfall (ES) are commonly used as measures of financial risk. VaR estimates the maximum possible loss in portfolio value over a specified period with a given confidence level, while ES gives an average loss for situations worse than the VaR estimate, thus addressing extreme market scenarios.

Historically, traditional Monte Carlo methods such as Geometric Brownian Motion (GBM) simulations and historical bootstrapping have been standard approaches for estimating these risks. These methods typically depend on historical data and simplifying assumptions, such as the normal distribution and stable market conditions. However, real-world financial markets often exhibit characteristics like heavy tails, sudden volatility changes, and structural shifts, which traditional methods may struggle to accurately capture.

Recently, machine learning (ML) techniques have gained attention for their potential to address these limitations. ML methods, particularly Generative Adversarial Networks (GANs) and Deep Quantile Regression (DQR), offer a more flexible approach capable of modeling complex and nonlinear financial patterns. In this report, we combine traditional Monte Carlo methods with these advanced ML techniques to examine their performance in estimating VaR and ES across various asset classes, including the SP 500, NASDAQ, and Bitcoin.

We perform thorough backtesting using Kupiec's unconditional coverage test and Christoffersen's conditional coverage test to evaluate how accurately each method predicts risk. Our analysis compares traditional and ML-based methods, highlighting their respective strengths and weaknesses, to provide practical insights for improved financial risk management.

2 Data Preparation

Daily price data for each asset was downloaded, cleaned of missing values, and aligned to a common calendar. This is important since BTC data exists for the whole week, while data for the S&P500 and NASDAQ is absent on the weekends. Log returns were computed. In addition to raw returns, a winsorized version was created to manage the impact of extreme outliers (This winsorized version is used specifically for Parametric Simulation using Geometric Brownian Motion).

3 Methodology

3.1 Traditional Monte Carlo Methods

3.1.1 Parametric Simulation (Geometric Brownian Motion)

This method assumes that asset returns follow a Geometric Brownian Motion (GBM) process. This is grounded in the assumption that log returns are normally distributed, with parameters being estimated from the historical data. More specifically, the drift and volatility are computed from daily historical log returns, and then used to simulate a large number of future price paths. These return paths are simulated using a series of random shocks taken from a normal distribution parameterized by the previously estimated drift and volatility terms. These shocks are then applied to an initial asset price over a specified forecast horizon.

This method is computationally efficient and elegant to implement and interpret, however, its simplifying assumptions lead to some limitations. Real world financial returns often exhibit skewness, kurtosis and volatility clustering that the GBM model cannot capture. As a result of this, it may underestimate the likelihood of extreme outcomes in periods of market stress, and thus underestimate risk. The formula used for GBM is specified below:

$$S_{t+\Delta t} = S_t \cdot e^{(\mu - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}Z} \quad (1)$$

where $Z \sim \mathcal{N}(0, 1)$. Each asset was simulated using 10,000 paths, and 10 sample paths for each asset were plotted out.

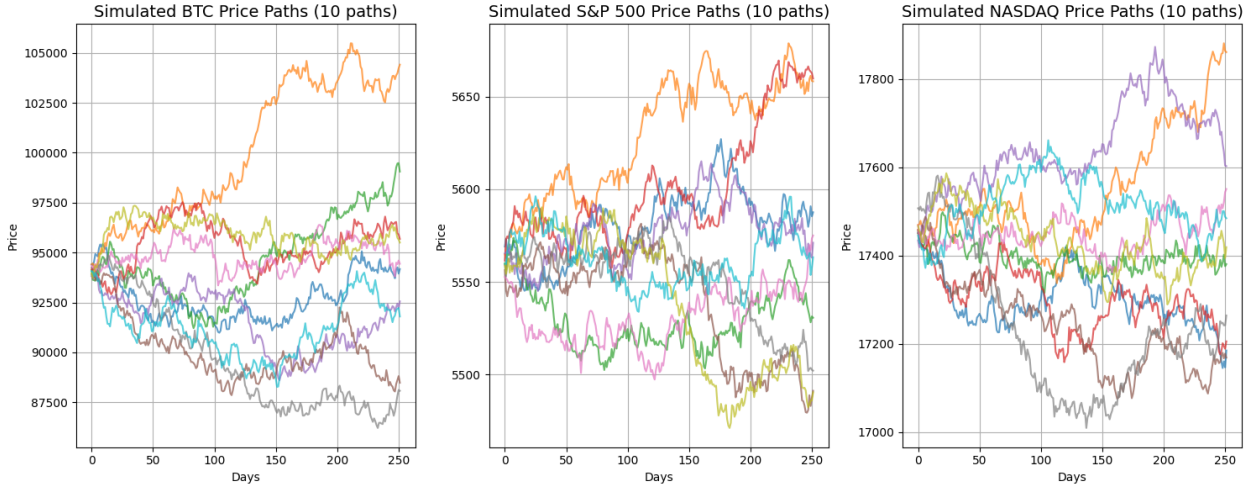


Figure 1: Plot of 10 paths for each ticker (GBM)

Table 1: GBM Monte-Carlo - 95% VaR and ES

Asset	VaR (95%)	ES (95%)
S&P500	-1.76%	-2.23%
NASDAQ	-2.18%	-2.70%
BTC-USD	-6.62%	-8.23%

3.1.2 Historical Bootstrapping

The bootstrapped Monte-Carlo method avoids any underlying assumptions about the returns distribution, instead generating simulated paths by resampling historical log returns with replacement. This preserves the empirical distribution of the returns, including heavy tails, skewness, or any other dependencies that the GBM simulation might lose. Bootstrapping, therefore, better reflects actual market behavior, especially in scenarios where the historical data exhibits a significant departure from normalcy. This is especially relevant in our data, where 2017 and 2020 are periods with extreme observations (especially for BTC).

Flexibility is the greatest strength of this method, but it does come with some tradeoffs. The assumption that past return patterns are representative of the future may not hold, especially for rapidly evolving markets like crypto. it also tends to be more sensitive to the sample size and quality of historical data, and can suffer from overfitting if the past includes unusual periods that are not expected to reoccur (once again, like 2020).

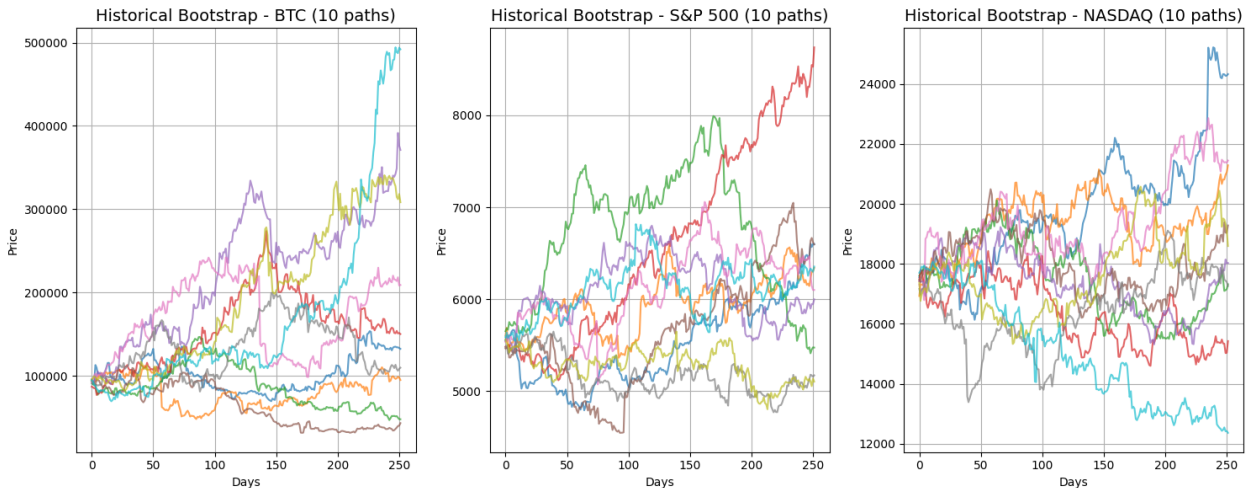


Figure 2: Plot of 10 paths for each ticker (Bootstrap)

Table 2: Bootstrap Monte-Carlo - 95% VaR and ES

Asset	VaR (95%)	ES (95%)
S&P500	-18.35%	-24.00%
NASDAQ	-21.43%	-27.65%
BTC-USD	-43.90%	-56.71%

3.2 Machine Learning Methods

3.2.1 Generative Adversarial Networks

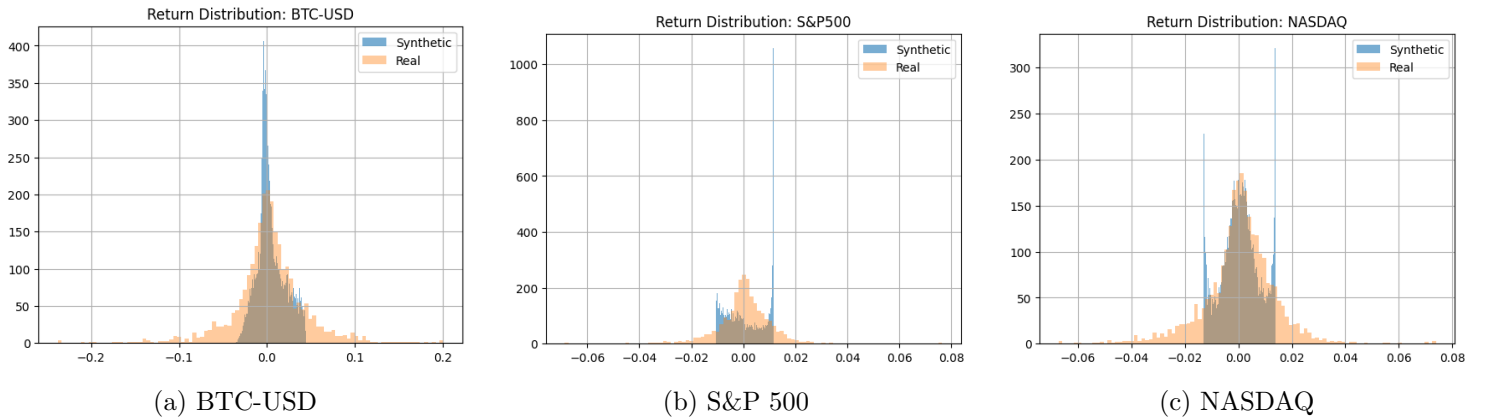
Generative Adversarial Networks (GANs) consist of two neural networks, the generator and the discriminator, which are trained simultaneously in a competitive manner.

The generator creates synthetic data designed to mimic real data distributions, while the discriminator evaluates whether the data presented to it is real or synthetic. During training, both networks improve iteratively, enhancing the generator’s ability to produce realistic synthetic data.

Static GAN. Before implementing a rolling-window procedure, we first trained a *single, static* Generative Adversarial Network on the entire sample of winsorised daily returns (2015–2025). The goal was to check whether a GAN, trained once, can reproduce the unconditional return distribution of each asset. Figure 3 illustrates the synthetic (blueblue) versus realised (orangeorange) histograms. Visually, the model captures the general shape of the empirical distribution, although the synthetic tails for the S&P 500 and NASDAQ are noticeably thinner. From each static GAN we drew 10 000 one-day returns and estimated the 95 % VaR and expected shortfall (ES). Results are reported in Table 3. Compared with the rolling GAN and DQR, the static model produces VaR values close to historical experience, but—because it never re-trains—cannot adapt to changing volatility regimes and therefore is *not* suitable for formal back-testing.

Table 3: Static GAN — 1-Day 95 % VaR and ES

Asset	VaR (95 %)	ES (95 %)
BTC-USD	−1.96%	−2.42%
S&P500	−1.00%	−1.03%
NASDAQ	−1.19%	−1.25%

Figure 3: Synthetic (blueblue) vs. real (orangeorange) return distributions from the *static* GAN.

Rolling GAN for VaR. To obtain day-by-day risk forecasts, we adopted a rolling approach: every 20 trading days the GAN is retrained on the latest 252 observations, generates 10,000 synthetic one-day returns, and the 5% quantile of this sample becomes the 1-day VaR. Backtesting shows the rolling GAN tends to produce overly conservative VaR levels (coverage rates around 25–30% versus an expected 5%), leading Kupiec’s test to reject correct unconditional coverage, though Christoffersen’s test suggests breaches are not clustered.

3.2.2 Deep Quantile Regression

Deep Quantile Regression (DQR) leverages neural networks to directly estimate quantiles from financial time series data, offering a straightforward approach to quantifying VaR without explicitly modeling the entire return distribu-

tion. By training a neural network to predict the conditional 5% quantile of returns, DQR are expected to provide direct forecasts of VaR.

While DQR offers simplicity and interpretability, we faced difficulties related to its conservative predictions. During our rolling-window backtesting procedure, DQR consistently overestimated the magnitude of potential losses, resulting in significantly higher breach rates compared to theoretical expectations. This indicates a potential need for better calibration or alternative modeling strategies to improve practical performance.

Table 4: Rolling DQR Back-test Summary (1-Day 95 % VaR)

Asset	Coverage Rate	Expected	Min VaR	Median VaR	Breaches	Total Obs.
BTC-USD	23.48%	5.00%	-11.58%	-2.12%	27	115
S&P500	24.35%	5.00%	-5.18%	-0.45%	28	115
NASDAQ	27.83%	5.00%	-3.24%	-0.63%	32	115

4 Risk Measures: VaR and ES

Value at Risk (VaR) and Expected Shortfall (ES) were computed from both the GBM simulation and the Bootstrapping simulation.

- **Value at Risk (VaR)** is a risk measure that estimates the maximum potential loss of a portfolio over a given time horizon at a specified confidence level. For example, a daily 95% VaR of \$1 million means that under normal market conditions, the portfolio is not expected to lose more than \$1 million on 95% of trading days.
- **Expected Shortfall (ES)**, also known as Conditional VaR, complements VaR by estimating the *average* loss on days when losses exceed the VaR threshold. It reflects the tail of the loss distribution and is considered a more coherent and conservative risk measure.

To compute Value at Risk (VaR) and Expected Shortfall (ES), we used two Python functions that operate directly on simulated return distributions. For a given confidence level α , the VaR is computed as the α -quantile of the distribution of simulated portfolio returns. This quantile represents the threshold below which the worst $\alpha\%$ of losses fall. The Expected Shortfall is then calculated as the average of all simulated losses that exceed the VaR threshold—that is, the mean of the lower $\alpha\%$ tail. For both the Geometric Brownian Motion (GBM) and historical bootstrapping models, we generated a large number of future return paths and applied these functions to obtain risk estimates across assets. This approach leverages Monte Carlo simulation to capture a wide range of potential outcomes and better reflect non-linear risk dynamics. For the simulations conducted in this report, we have taken $\alpha = 5$.

VaR Visualization (Traditional Monte-Carlo Methods)

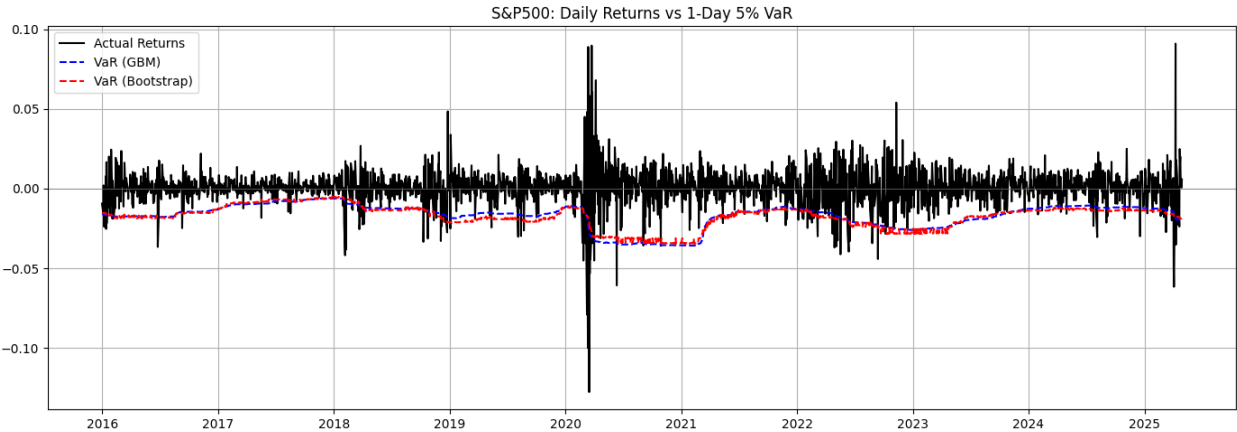


Figure 4: Illustrative plot of S&P500 returns with VaR line for both methods

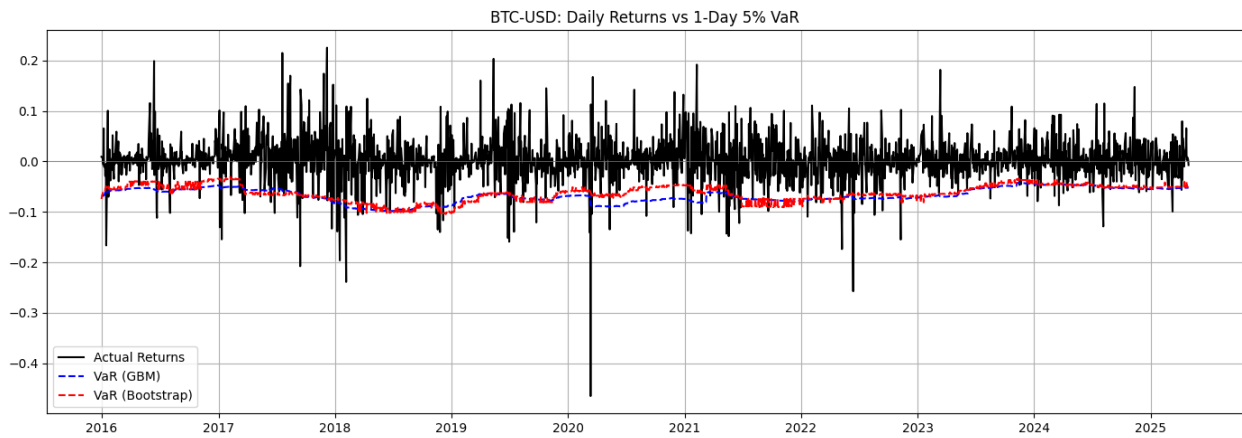


Figure 5: Illustrative plot of BTC returns with VaR line for both methods

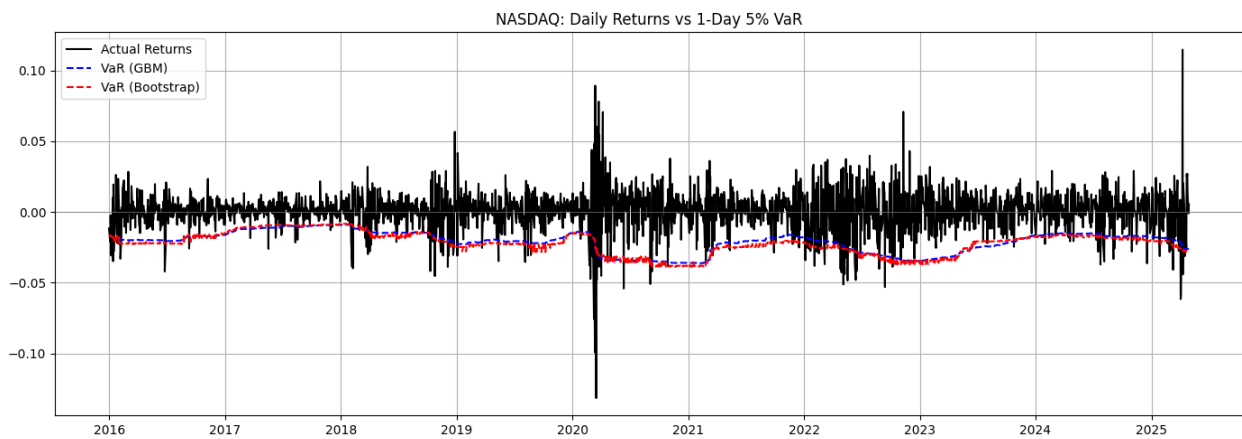


Figure 6: Illustrative plot of NASDAQ returns with VaR line for both methods

5 Backtesting and Model Validation

5.1 Kupiec's Unconditional Coverage Test

Kupiec's test, also known as the Proportion of Failures (POF) test, evaluates whether the number of Value-at-Risk (VaR) breaches observed in actual returns aligns with the expected number of breaches.

- **Reason for Usage:** This test assesses whether the frequency of breaches matches the specified VaR level (e.g., 5%).
- **Interpretation of results:**
 - A **high p-value** (typically > 0.05) suggests that the model's breach frequency is consistent with the expected rate.
 - A **low p-value** indicates that the model may either underestimate or overestimate the level of risk.

5.2 Christoffersen's Conditional Coverage Test

Christoffersen's test extends Kupiec's by additionally checking whether the breaches are independently distributed over time, or whether they cluster.

- **Purpose:** To test for both correct coverage and independence of VaR breaches (i.e., whether breaches are randomly scattered or clustered).
- **Interpretation of results:**
 - A **high p-value** suggests that VaR breaches are not serially correlated.
 - A **low p-value** may indicate time-dependence or clustering in the model's errors.

5.3 Implementation in Python

The backtesting functions implemented in Python perform these tests using binary sequences of VaR breaches:

- `kupiec_test(breaches, alpha):`
 - Takes in an array of 0s and 1s indicating whether a VaR breach occurred on each day.
 - Computes the likelihood under the null (correct breach frequency) and observed frequency.
 - Returns the test statistic and p-value.
- `christoffersen_test(breaches):`
 - Analyzes transition probabilities between consecutive breach states.
 - Tests for independence using a likelihood ratio test.
 - Returns the test statistic and p-value for conditional coverage.

These tests were applied to both the GBM-based and bootstrapping VaR models across all three assets (S&P500, NASDAQ, and BTC) to assess model validity.

5.4 Traditional Monte Carlo Results

Table 6 and Table 7 report Kupiec’s unconditional-coverage and Christoffersen’s conditional-coverage statistics for the rolling GBM and Bootstrapping methods, respectively. Table 5 provides the Breach Rates and Deviation % for each method and asset.

Table 5: VaR Breaches Summary

Model	Asset	Total Days	VaR Breaches	Breach Rate (%)	Expected Rate (%)	Deviation (%)
GBM	S&P500	2345	136	5.80	5.00	0.80
GBM	BTC-USD	2345	107	4.56	5.00	-0.44
GBM	NASDAQ	2345	153	6.52	5.00	1.52
Bootstrap	S&P500	2345	127	5.42	5.00	0.42
Bootstrap	BTC-USD	2345	119	5.07	5.00	0.07
Bootstrap	NASDAQ	2345	134	5.71	5.00	0.71

Table 6: GBM Back-test Summary

Asset	Kupiec LR	Kupiec p	Christoff. LR	Christoff. p
S&P500	3.008	0.083	17.754	2.514e-05
BTC-USD	0.970	0.325	0.262	0.608
NASDAQ	10.512	0.001	6.128	0.013

Table 7: Bootstrap Back-test Summary

Asset	Kupiec LR	Kupiec p	Christoff. LR	Christoff. p
S&P500	0.832	0.362	16.960	3.818e-05
BTC-USD	0.027	0.869	0.646	0.422
NASDAQ	2.413	0.120	4.871	0.027

5.5 Machine Learning Methods Results

Table 8 and Table 9 report Kupiec’s unconditional-coverage and Christoffersen’s conditional-coverage statistics for the rolling GAN and DQR VaR forecasts, respectively.

In comparison with the traditional Monte-Carlo results, both ML models require calibration to achieve realistic tail-probability control.

Table 8: Rolling GAN Back-test Summary (1-Day 95 % VaR)

Asset	Coverage	Kupiec LR	Kupiec p	Christoff. LR	Christoff. p	Breaches / N
BTC-USD	39.83%	130.215	0.0000	3.036	0.0814	47 / 118
S&P500	35.59%	105.792	0.0000	1.225	0.2685	42 / 118
NASDAQ	35.59%	105.792	0.0000	0.938	0.3328	42 / 118

Table 9: Rolling DQR Back-test Summary (1-Day 95 % VaR)

Asset	Coverage	Kupiec LR	Kupiec p	Christoff. LR	Christoff. p	Breaches / N
BTC-USD	24.35%	49.023	0.0000	0.201	0.6541	28 / 115
S&P500	24.35%	49.023	0.0000	0.315	0.5747	28 / 115
NASDAQ	32.17%	85.206	0.0000	0.750	0.3866	37 / 115

6 Performance of Traditional Monte-Carlo Methods

This section collates the findings for the two Monte-Carlo approaches employed in this study (GBM and Bootstrapping). It explains why the models performed best overall on BTC-USD, and worst overall on NASDAQ. Finally, it suggests some improvements that could be made.

6.1 Diagnosis of Test Results

1. S&P 500:

- GBM passes Kupiec’s test, however, it fails on Christoffersen’s test. This indicates that the model is underestimating volatility in times of stress. Due to this, the average number of breaches is acceptable, but we see a large number of breaches in a row during times of crisis.
- The bootstrap model performs roughly the same, passing Kupiec’s test and failing Christoffersen’s. Therefore, it exhibits the same problems in times of stress.

2. BTC-USD:

- GBM offers good results across the board, passing both Kupiec’s and Christoffersen’s tests. This indicates that the model is accurately estimating the number of expected breaches, and not having them clustered too close together.
- Bootstrapping also leads to similarly acceptable results.

3. NASDAQ:

- GBM fails entirely here, failing both tests in terms of LR and in terms of p-value. Therefore, the model is not able to accurately predict the frequency of breaches, nor their timing.
- The bootstrap model performs better, passing Kupiec’s test. This indicates that the breach frequency is acceptable overall. However, it still fails Christoffersen’s test, meaning that the model is still not representative in times of crisis

6.2 Possible Causes

- The Monte-Carlo models performed the best on BTC, and the worst on NASDAQ. BTC’s return series, though volatile, exhibited comparatively less clustered extreme return during the sample period, making it friendlier to the models’ simplifying assumptions.
- In contrast, NASDAQ’s tech heavy composition may lead to higher volatility, stronger clustering, and fatter tailed return distributions. All these factors violate the models’ simplifying assumptions.
- The S&P 500 offers a middle ground, with its broadly diverse index spanning multiple sectors, showing more stable volatility and less extreme clustering.

6.3 Possible Improvements

- **Time-Varying Volatility Models:** Models such as GARCH could improve VaR estimates by accounting for volatility clustering.
- **Filtered Historical Simulation:** This could improve the bootstrapping method by scaling past returns according to current volatility.
- **Jump Diffusion Models:** This could be more suitable for assets like NASDAQ, where price jumps are more common.

7 Performance of Machine-Learning VaR Models

This section synthesises the key findings for the two machine-learning approaches employed in this study—**Generative Adversarial Networks (GANs)** and **Deep Quantile Regression (DQR)**—and explains why both models produced excessively conservative 1-day 95 % VaR forecasts.

7.1 Observed Over-Coverage

Table 8 (GAN) and Table 9 (DQR) show breach frequencies far above the regulatory 5 % benchmark:

- **GAN (rolling):** empirical coverage rates range from **35–40 %** (roughly $8\times$ the target).
- **DQR (rolling):** coverage ranges from **24–32 %** ($5\text{--}6\times$ the target).

Both methods therefore draw VaR lines that are *too low*, causing an excessive number of breaches.

7.2 Statistical Evidence

- **Kupiec’s unconditional-coverage test** rejects the null H_0 : coverage = 5% for all assets (p -values ≈ 0).
- **Christoffersen’s conditional-coverage test** yields p -values > 0.08 (GAN) and > 0.38 (DQR), indicating that breaches, while far too frequent, are not clustered in time. The models are *biased*, not dynamically unstable.

7.3 Root Causes

Data-level factors

1. *High-volatility episodes:* 2017 and 2020 introduce extreme left-tail observations (especially for BTC), over-weighting negative returns in a 252-day window.
2. *Sample size vs. window length:* 252 observations are marginal for a GAN and leave DQR’s 5 % quantile dominated by a handful of extremes.

Model-level factors

- **GAN.** Mode collapse and lack of regularisation can elongate tails; a strong discriminator squeezes central mass, leaving a thick synthetic left tail.
- **DQR.** Single pinball loss at $q = 0.05$ encourages the network to undershoot: it is “safer” (in loss terms) to predict a quantile slightly below the true one.

Workflow factors

1. Potential scaling drift if `StandardScaler` is fit on windows containing extreme values.
2. High learning rates / strong dropout amplify GAN instability and push DQR predictions downward.

7.4 Practical Remedies

- **Post-hoc calibration.** Rescale raw VaR by a factor $c = \alpha/p_{\text{emp}}$ so that empirical coverage matches the 5% target.
- **Feature enrichment.** Feed rolling volatility, GARCH σ , or macro-regime indicators into GAN/DQR to capture evolving risk.
- **Multi-quantile training.** Predict 5%, 50%, 95% simultaneously; the intermediate quantiles regularise the left tail.
- **Stabilised GAN variants.** Use Wasserstein GAN with gradient penalty, label smoothing, or longer training windows to reduce tail explosion.

8 Conclusion

The traditional Monte-Carlo methods performed best on BTC, possibly because of the wide time window used, and also performed well on the S&P 500. However, their performance on NASDAQ indicates that further refinements may be needed to account for assets with higher volatility clustering and fatter tails.

Under the present configuration, both ML models are *far more conservative* than required, whereas traditional Monte-Carlo methods delivered VaR thresholds closer to the regulatory benchmark. ML models remain attractive for capturing non-Gaussian features, but must be *carefully calibrated and feature-rich* to deliver usable tail-risk estimates in practice.